从功能点及其关联关系到功能逻辑架构数据

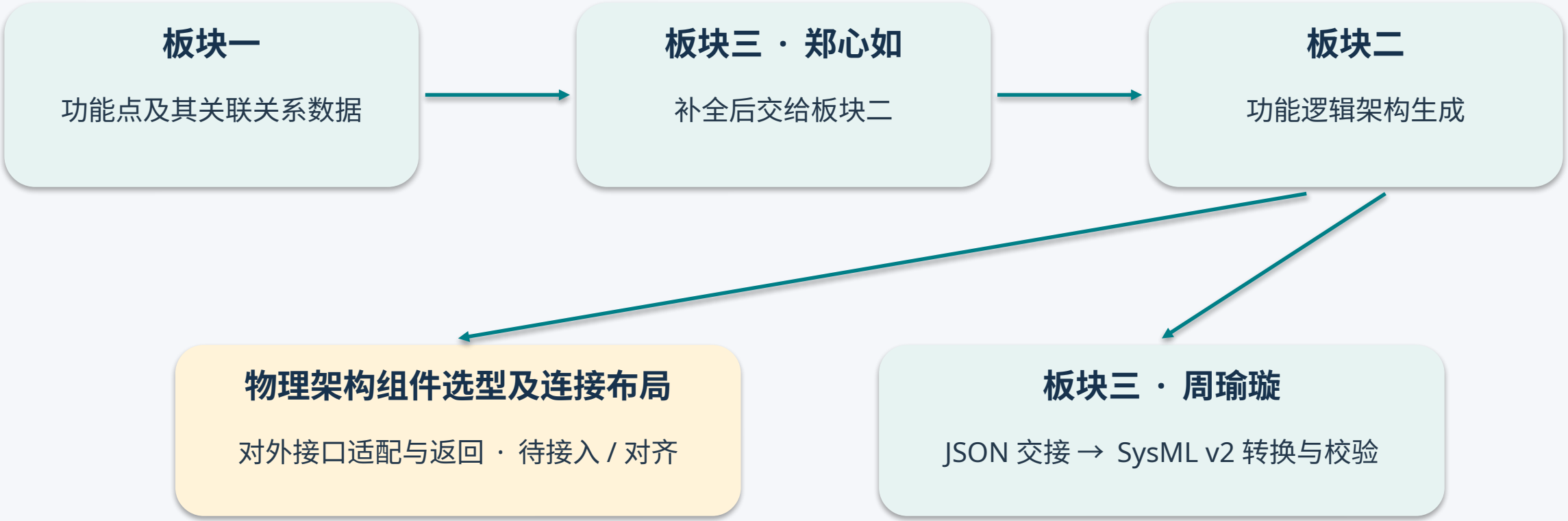
板块二
研究进展与接口设计

BIRCH 功能聚类 / 功能关系分析与拓扑连接生成

汇报重点：处理能力、数据交付、两个对外接口及当前待办

2026 年 10 月 28 日

上游由郑心如补全，下游分为接口对接与模型转换



板块二负责功能逻辑架构 JSON； SysML v2 转换与工具校验由周瑜璇负责。

研究进展覆盖输入、聚类、连接和数据输出

环节	已有成果	仍需推进
输入与校验	MBSE_app 前置校验、关系类型标准化	统一对外 Schema 与错误定位
功能聚类	TF-IDF + BIRCH 形成归属数据	特征组合、阈值与业务解释性评价
关系与拓扑	挂接源 / 目标簇，生成端口与流对象	端口类型 / 单位及多端口匹配
输出与集成	Excel / JSON 输出及校验报告	两个对外接口与字段协商

本部分依据 MBSE_app；接口章节另依据需求 MD 和服务器版本说明。

技术路线：先聚类，再组织关系与拓扑连接

01 输入规范与校验

ID 、引用、关系与端口

02 语义表征

分词 → TF-IDF → L2

03 BIRCH 功能聚类

得到功能 ID— 簇 ID 映射

04 归属与拓扑组织

关系簇标签 / 端口 / 流对象

05 质量评价

内聚 / 解耦 / 碎片化

06 数据交付

功能逻辑架构 JSON

当前功能关系参与校验、标注和连接生成，尚未作为 BIRCH 的图约束。

前置校验：先检查输入，再生成功能逻辑架构数据

输入合法性与引用检查

```
1 usage
def step1b_dedup(self, df_rel):
    """步骤1b：关系图去重"""
    before = len(df_rel)
    df_dedup = df_rel.drop_duplicates(
        subset=["源功能点ID", "目标功能点ID", "关系类型"], keep="first"
    ).reset_index(drop=True)
    after = len(df_dedup)
    if before != after:
        self._info(f"步骤1-去重：关系图 {before} → {after} 行（去除{before-after}条重复）")
    return df_dedup

1 usage
def step2_id_reference(self, df_func, df_rel):
    """步骤2：ID引用完整性检查"""
    step = "步骤2-ID引用完整性"
    valid_ids = set(df_func["功能点ID"].astype(str).tolist())
    for i, row in df_rel.iterrows():
        sid = str(row.get("源功能点ID", "")).strip()
        tid = str(row.get("目标功能点ID", "")).strip()
        if sid and sid not in valid_ids:
            self._err(step, msg: f"关系图第{i+2}行：源功能点ID不存在 -> {sid}")
        if tid and tid not in valid_ids:
            self._err(step, msg: f"关系图第{i+2}行：目标功能点ID不存在 -> {tid}")
```

已有运行结果：前置校验通过

```
MBSE 前置完整性校验报告
========================================

【数据清理记录】

【警告（不阻断流程）】

【致命错误】
  （无）

【结论】
  ✅ 前置校验全部通过
```

校验通过表示输入满足计算条件，不等于功能需求完整或工程架构正确。

聚类核心： TF-IDF 表征与 BIRCH 功能聚类

```
# ----------TF-IDF+BIRCH聚类----------
df_func["seg_text"] = df_func["功能点名称"].apply(seg_and_filter)
tfidf = TfidfVectorizer()
tfidf.fit(df_func["seg_text"].tolist())
X_raw = tfidf.transform(df_func["seg_text"])
X = normalize(X_raw, norm="l2")

birch = Birch(threshold=BIRCH_THRESHOLD, branching_factor=BRANCHING_FACTOR, n_clusters=None)
labels = birch.fit_predict(X)
df_func["功能簇ID"] = labels
```

原型特征：功能点名称 → jieba 分词 → TF-IDF → L2 归一化

MBSE_app 参数： threshold=0.6； branching_factor=50； n_clusters=None 。

已有原型输出可展示功能点到功能簇的归属

7
功能点

6
功能簇

13
关系记录

13
连接记录

【表1】聚类架构分层表

功能点ID	功能点名称	架构层级	子系统名称	功能簇名称	功能簇ID
4b84254e-617e-4ffc-b8fd-4469d1415a63	检测动力电池剩余电量	组件层	BMS电池管理子系统	动力电池状态检测功能	0
cc88c882-2086-4dc5-a16b-398330cf977c	限制驱动功率	组件层	功率控制子系统	驱动功率限制控制功能	1
48598659-f879-45b1-beb7-2921717e30dc	提示驾驶员充电	组件层	整车交互子系统	驾驶员充电提示功能	2
6a1c3df2-f322-41ff-bc36-24d531f948fb	允许正常动力输出	组件层	功率控制子系统	驱动功率限制控制功能	3
80107eef-d7ae-48a0-9a93-2fb38db012ed	需求与功能输入管理	组件层	架构生成子系统	需求与功能输入管理功能	4
d00eb5ef-c5f2-419c-b482-1dd7b0b744bd	物理组件库与历史架构模型库的导入	组件层	架构生成子系统	架构知识库管理与方案生成功能	5
fe237e5d-7415-43ad-b90c-94b7d4aa2cb5	多物理架构方案生成	组件层	架构生成子系统	架构知识库管理与方案生成功能	5

样例中多数簇较小，需要结合碎片化指标与业务复核，不能仅以“生成成功”判断质量。

关系处理核心：有效端点、标准关系与所属功能簇

端点过滤与关系类型处理

```
for _, rr in df_rel.iterrows():
    src_id = rr["源功能点ID"]
    tgt_id = rr["目标功能点ID"]
    #----------核心过滤：只保留两端都在聚类结果中的关系------------
    if src_id not in valid_func_ids or tgt_id not in valid_func_ids:
        filtered_count += 1
        continue
     #保留这条关系
    filtered_relations.append(rr)

    raw_type = rr.get("关系类型", "data_flow")
    rel_code = normalize_relation_code(raw_type)
    conf = rr.get("置信度", 0.9)
    try:
        conf = float(conf)
        if math.isnan(conf):
            conf = 0.9
    except (TypeError, ValueError):
        conf = 0.9
    evidence = rr.get("证据描述", "")
    if pd.isna(evidence):
        evidence = ""
```

源 / 目标簇关联与记录组织

```
    #从聚类结果中查找源和目标所属的簇ID
    src_cluster = fid_to_cluster.get(src_id, "未知")
    tgt_cluster = fid_to_cluster.get(tgt_id, "未知")

    table2_rows.append({
        "映射关系ID": f"REL-{rel_seq:03d}",
        "源功能点ID": src_id,
        "源功能点名称": rr["源功能点名称"],
        "目标功能点ID": tgt_id,
        "目标功能点名称": rr["目标功能点名称"],
        "关系类型(标准编码)": rel_code,
        "关系中文名称": get_relation_cn(rel_code),
        "方向": get_relation_direction(rel_code),
        "置信度": conf,
        "证据描述": evidence,
        "所属源簇": f"簇{src_cluster}",
        "所属目标簇": f"簇{tgt_cluster}"
    })
    rel_seq += 1

df_table2 = pd.DataFrame(table2_rows)
print(f"表2生成完成：保留 {len(df_table2)} 条关系，过滤掉 {filtered_count} 条（端点不在聚类结果中）")
```

原型以前置校验保证引用有效；生成阶段保留端点过滤，关系记录携带簇归属与证据。

关系运行结果：功能关系附加源簇与目标簇信息

==========

【表2】聚类功能点关系表

==========

映射关系ID	源功能点ID	源功能点名称	目标功能点ID	目标功能点名称	关系类型(标准
REL-001	4b84254e-617e-4ffc-b8fd-4469d1415a63	检测动力电池剩余电量	cc88c882-2086-4dc5-a16b-398330cf977c	限制驱动功率	DATA_FLOW	数据
REL-002	4b84254e-617e-4ffc-b8fd-4469d1415a63	检测动力电池剩余电量	48598659-f879-45b1-beb7-2921717e30dc	提示驾驶员充电	DATA_FLOW	数据
REL-003	4b84254e-617e-4ffc-b8fd-4469d1415a63	检测动力电池剩余电量	6a1c3df2-f322-41ff-bc36-24d531f948fb	允许正常动力输出	DATA_FLOW	数
REL-004	4b84254e-617e-4ffc-b8fd-4469d1415a63	检测动力电池剩余电量	cc88c882-2086-4dc5-a16b-398330cf977c	限制驱动功率	TRIGGER	事件触
REL-005	4b84254e-617e-4ffc-b8fd-4469d1415a63	检测动力电池剩余电量	48598659-f879-45b1-beb7-2921717e30dc	提示驾驶员充电	TRIGGER	事件
REL-006	4b84254e-617e-4ffc-b8fd-4469d1415a63	检测动力电池剩余电量	6a1c3df2-f322-41ff-bc36-24d531f948fb	允许正常动力输出	TRIGGER	事件
REL-007	cc88c882-2086-4dc5-a16b-398330cf977c	限制驱动功率	48598659-f879-45b1-beb7-2921717e30dc	提示驾驶员充电	CONCURRENT	并发
REL-008	cc88c882-2086-4dc5-a16b-398330cf977c	限制驱动功率	6a1c3df2-f322-41ff-bc36-24d531f948fb	允许正常动力输出	CONSTRAINT	约束限
REL-009	48598659-f879-45b1-beb7-2921717e30dc	提示驾驶员充电	6a1c3df2-f322-41ff-bc36-24d531f948fb	允许正常动力输出	CONSTRAINT	约束限
REL-010	80107eef-d7ae-48a0-9a93-2fb38db012ed	需求与功能输入管理	fe237e5d-7415-43ad-b90c-94b7d4aa2cb5	多物理架构方案生成	DATA_FLOW	数
REL-011	d00eb5ef-c5f2-419c-b482-1dd7b0b744bd	物理组件库与历史架构模型库的导入	fe237e5d-7415-43ad-b90c-94b7d4aa2cb5	多物理架构方案生成	DATA_FLOW	
REL-012	80107eef-d7ae-48a0-9a93-2fb38db012ed	需求与功能输入管理	d00eb5ef-c5f2-419c-b482-1dd7b0b744bd	物理组件库与历史架构模型库的导入	TRIGGER	
REL-013	80107eef-d7ae-48a0-9a93-2fb38db012ed	需求与功能输入管理	4b84254e-617e-4ffc-b8fd-4469d1415a63	检测动力电池剩余电量	DATA_FLOW	数

文档所附案例含 13 条关系；完整保留原截图，右缘未完整显示的字段以原 JSON/Excel 为准。

拓扑连接核心：关联端口、流对象与所属功能簇

关系类型与流对象处理

```
for rr in filtered_relations:
    src_fid = rr["源功能点ID"]
    tgt_fid = rr["目标功能点ID"]
    src_name = fid_to_name[src_fid]
    tgt_name = fid_to_name[tgt_fid]
    rel_code = normalize_relation_code(rr.get("关系类型", "data_flow"))

    src_outputs = fid_io[src_fid]["outputs"]
    tgt_inputs = fid_io[tgt_fid]["inputs"]
    #端口匹配核心逻辑：取第一个输出/输入
    src_port = src_outputs[0] if len(src_outputs) > 0 else "—"
    tgt_port = tgt_inputs[0] if len(tgt_inputs) > 0 else "—"

    if rel_code == "DATA_FLOW":
        flow_obj = src_port
    elif rel_code == "TRIGGER":
        flow_obj = "触发信号"
    else:
        flow_obj = "—"

    src_cluster = fid_to_cluster[src_fid]
    tgt_cluster = fid_to_cluster[tgt_fid]
```

连接记录与聚类归属

```
    table3_rows.append({
        "连接ID": f"L-{conn_seq:03d}",
        "源功能点名称": src_name,
        "源端口(取自outputs)": src_port,
        "目标功能点名称": tgt_name,
        "目标端口(取自inputs)": tgt_port,
        "关系类型": rel_code,
        "流对象": flow_obj,
        "所属源簇": f"簇{src_cluster}",
        "所属目标簇": f"簇{tgt_cluster}"
    })
    conn_seq += 1
df_table3 = pd.DataFrame(table3_rows)
print(f"表3生成完成：基于过滤后的 {len(df_table3)} 条连接")
```

原型采用首项端口策略；后续需完善类型、单位、多端口匹配及待确认标记。

拓扑运行结果：端口、流对象与功能簇可追溯

【表3】IBD/BDD拓扑连接表-逻辑层面

连接ID	源功能点名称	源端口(取自outputs)	目标功能点名称	目标端口(取自inputs)	关系类型	流对象	所属源簇	所属目标簇
L-001	检测动力电池剩余电量	动力电池剩余电量	限制驱动功率	动力电池剩余电量	DATA_FLOW	动力电池剩余电量	簇0	簇1
L-002	检测动力电池剩余电量	动力电池剩余电量	提示驾驶员充电	动力电池剩余电量	DATA_FLOW	动力电池剩余电量	簇0	簇2
L-003	检测动力电池剩余电量	动力电池剩余电量	允许正常动力输出	动力电池剩余电量	DATA_FLOW	动力电池剩余电量	簇0	簇3
L-004	检测动力电池剩余电量	动力电池剩余电量	限制驱动功率	动力电池剩余电量	TRIGGER	触发信号	簇0	簇1
L-005	检测动力电池剩余电量	动力电池剩余电量	提示驾驶员充电	动力电池剩余电量	TRIGGER	触发信号	簇0	簇2
L-006	检测动力电池剩余电量	动力电池剩余电量	允许正常动力输出	动力电池剩余电量	TRIGGER	触发信号	簇0	簇3
L-007	限制驱动功率	驱动功率限制指令	提示驾驶员充电	动力电池剩余电量	CONCURRENT	—	簇1	簇2
L-008	限制驱动功率	驱动功率限制指令	允许正常动力输出	动力电池剩余电量	CONSTRAINT	—	簇1	簇3
L-009	提示驾驶员充电	充电提示信息	允许正常动力输出	动力电池剩余电量	CONSTRAINT	—	簇2	簇3
L-010	需求与功能输入管理	标准化需求功能结构化数据	多物理架构方案生成	标准化需求功能数据	DATA_FLOW	标准化需求功能结构化数据	簇4	簇5
L-011	物理组件库与历史架构模型库的导入	可用知识库索引与实例数据	多物理架构方案生成	标准化需求功能数据	DATA_FLOW	可用知识库索引与实例数据	簇5	簇5
L-012	需求与功能输入管理	标准化需求功能结构化数据	物理组件库与历史架构模型库的导入	物理组件库文件	TRIGGER	触发信号	簇4	簇5
L-013	需求与功能输入管理	标准化需求功能结构化数据	检测动力电池剩余电量	车辆启动信号	DATA_FLOW	标准化需求功能结构化数据	簇4	簇0

对应 13 条连接；数据流、触发及并发 / 约束保持不同流对象表达。

输出核心代码： Excel 与 JSON 两种格式

Excel

```python
# 【阶段F】输出Excel
# ============================================================
excel_out = os.path.join(OUTPUT_DIR, "MBSE_三张输出表.xlsx")
try:
    with pd.ExcelWriter(excel_out, engine="openpyxl") as writer:
        df_table1.to_excel(writer, sheet_name="聚类架构分层表", index=False)
        df_table2.to_excel(writer, sheet_name="聚类功能点关系表", index=False)
        df_table3.to_excel(writer, sheet_name="IBD_BDD拓扑连接表-逻辑层面", index=False)
    print(f"✅ Excel输出成功：{excel_out}")
except Exception as e:
    print("❌ 写Excel出错，请关闭已经打开的输出文件！")
    print(repr(e))
```

JSON

```python
# 【阶段G】输出JSON
# ============================================================
json_export = {
    "聚类架构分层表": clean_df_for_json(df_table1),
    "聚类功能点关系表": clean_df_for_json(df_table2),
    "IBD_BDD拓扑连接表-逻辑层面": clean_df_for_json(df_table3),
    "聚类评估指标": clean_nan(metrics),
}
json_path = os.path.join(OUTPUT_DIR, "MBSE_三张输出表.json")
with open(json_path, "w", encoding="utf-8") as f:
    json.dump(json_export, f, ensure_ascii=False, indent=2, allow_nan=False)
print(f"✅ JSON输出成功：{json_path}")
```

Excel 按 Sheet 保存结果； JSON 采用 UTF-8 及 allow_nan=False ，避免导出非标准数值。

输出运行截图：架构结果与前置校验报告已形成

生成文件清单：

1. MBSE_三张输出表.xlsx （3个Sheet）
2. MBSE_三张输出表.json （结构化数据+评估指标）
3. preflight_validation_report.txt （前置校验报告）

整体交付名称：功能逻辑架构数据

原型文件名保留原代码命名，后续按对接字段形成交换格式。

主线处理证据来自 MBSE_app ；两个对外接口的设计与联调另在接口章节说明。

产出统一为功能逻辑架构数据，按维度组织

维度	内容	用途
功能归属	功能 ID 、名称、簇、子系统及层级	组织功能簇与追溯
功能关系	端点、关系类型、置信度、来源、所属簇	表达功能及簇间关联
逻辑连接	源 / 目标端口、流对象、连接及所属簇	支撑拓扑分析与下游转换
评价信息	聚类数量、语义与几何指标	辅助判断聚类质量

JSON 一路对接物理架构模块（字段待对齐），另一路交周瑜璇进行 SysML v2 转换与校验。

两个对外接口对应需求 MD 的第 2 、第 3 项

接口一 · 逻辑架构生成

POST /generate_logical_arch

输入：上下文 JSON + 生成选项

输出：逻辑架构 JSON ，功能簇包含 function_id

接口二 · 输入补全

POST /complement_arch_gen_context

输入：上下文 + 提示文字 + 选项

输出：新上下文、是否需补全及说明

服务器接口：两个对外入口仍在开发和字段对齐；第 1 项资源规范化不属于本板块。

接口二：检查与补全分开，用户确认后纳入上下文

POST /complement_arch_gen_context

需求 MD 第 3 项	约定内容
请求	PhysicalArchGenContext + 可为空的用户提示 + options
only_check=true	只判断上下文完整性，不实际补全
only_check=false	进入补全处理；按对话设计结合用户反馈确认
响应	新的上下文 + 是否需要补全的布尔标志 + 补全说明

服务器已有候选预览与确认； only_check 为 MD 需求，尚需接入， API 选项后续统一。

接口伪代码：复用计算内核，按约定映射与返回

逻辑架构生成（待接入）

接收上下文与生成选项

校验并适配功能点及其关联关系

调用服务器内部聚类服务

按 cluster_id 组织 function_id 集合

按双方确认 Schema 返回 JSON

输入补全（功能设计）

接收上下文、用户提示及选项

若 only_check：仅判断完整性并返回

否则：生成建议并接收用户反馈

校验合并明确采纳及自行补充内容

返回新上下文、补全标志及说明

伪代码表达目标设计；服务器候选确认基础与完整对话补全、 only_check 需求须分别说明。

当前关键任务是字段对齐，而非仅增加接口路径

优先级	工作项	可验收结果
P0	与物理架构模块逐项对齐字段	共同 JSON 样例 + 字段映射清单
P0	补全建议与用户反馈合并	采纳 / 修改 / 拒绝可追溯，冲突有追问
P1	完善两个对外入口与会话管理	接口状态、上下文与输出一致
P1	验证特征、阈值和端口匹配	固定案例的指标、差异与人工复核记录
交付	向周瑜璇交接 JSON 并收集反馈	交接版本明确，转换所需字段可补齐

需讨论：功能簇结构、关系类型及方向、连接端点 ID 、来源字段、任务 / 版本与多轮增量规则。